Logging Book: Writing Good Log Messages and Structured Logging

Dev Practices Handbooks

February 4, 2026

Contents

Chapter 1

Why Logging Matters

Logging is how a system narrates events to the people who operate it. A good log message explains the event name, the outcome, and the key fields and IDs. It should do this fast, without drama. The goal is simple: make investigation cheaper.

1.1 What a Good Log Gives You

A strong log message helps you answer three questions:

- What event happened?
- What was the outcome and level?
- Which IDs or fields let me trace it?

When those answers are present, the message is actionable. When they are missing, every incident becomes a hunt.

1.2 The Core Vocabulary

Use consistent terms across the book and across services. This keeps team conversations short and precise.

Term	Meaning
Event name	A short verb phrase that describes the action, for example `job.completed`.
Level	The severity filter: debug, info, warn, error, or fatal.
Outcome	The result of the event: success, failure, or retry.
Fields	Structured data attached to the event.
IDs	Stable identifiers such as request id, correlation id, user id, or job id.

1.3 Checklist: The Minimum Message

- Event name is explicit.
- Outcome is stated.
- Level matches the outcome.
- IDs are present and stable.
- Fields cover the minimum context.

Chapter 2

Log Levels and Intent

Levels are filters, not decorations. They help people skim, and they help alerts route. A misused level creates noise or hides risk.

2.1 Default Mapping

Use a small set of levels. Keep meaning consistent across services.

Level	Use it when
DEBUG	You need internal state for diagnosis. Do not ship to production unless controlled.
INFO	The event is expected and the outcome is normal.
WARN	The outcome is unusual but the request still completes.
ERROR	The event failed or data could not be processed.
FATAL	The process cannot continue safely.

2.2 Outcome Drives Level

The outcome should drive the level. If the outcome is `success`, the level is usually info. If it is `failure`, use error. If the system recovered, use warn. Rare exceptions are fine, but they should be explicit.

2.3 Checklist: Level Hygiene

- Every error-level event has an outcome of `failure`.
- Warnings explain why recovery happened.
- Debug logs are safe to disable without hiding important events.
- Fatal logs are rare and always include a clear next step.

Chapter 3

Writing Clear Messages

A log message is a short report. It should say what happened and why it matters. The message is still useful even when fields are stripped or hidden.

3.1 Message Shape

Use a compact pattern: event name, outcome, and one strong noun. Example: `job.completed: success` or `http.call: failure`. That makes the first scan fast.

3.2 Prefer Concrete Words

Avoid vague words like `"unexpected"` or `"something"`. Name the event, the outcome, and the target. If you can name a field, do it.

3.3 Checklist: Message Review

- Event name and outcome are in the message.
- At least one ID appears in fields.
- The message is still meaningful without fields.
- No sensitive values appear in message or fields.

3.4 Python Example

```
import logging

logger = logging.getLogger("payments")

logger.info(
```

```
    "charge.attempt: outcome=success",
    extra={
        "event_name": "charge.attempt",
        "level": "info",
        "outcome": "success",
        "ids": {"request_id": "req_7f9a"},
        "fields": {
            "amount": 2400,
            "currency": "USD",
            "duration_ms": 185,
            "attempt": 1,
        },
    },
)
```

Chapter 4

Structured Logging

Structured logging separates the message from the fields. It makes logs searchable, joinable, and easy to aggregate. It also makes mistakes visible.

4.1 A Minimal Schema

Keep a consistent schema so tools and people can rely on it.

Field	Purpose
`event_name`	Short verb phrase for the event.
`level`	The severity filter.
`outcome`	success, failure, retry, or partial.
`ids`	Request, correlation, job, or user IDs.
`fields`	Event-specific details like `duration_ms`.

4.2 Stable Keys

Keep field names stable across services. Renaming keys breaks queries and dashboards. If you must change a key, support both for a migration period.

4.3 Python Example with `extra`

```
import logging

logger = logging.getLogger("pipeline")

logger.warning(
    "batch.load: outcome=retry",
    extra={
        "event_name": "batch.load",
```

```
        "level": "warn",
        "outcome": "retry",
        "ids": {"job_id": "job_103"},
        "fields": {
            "attempt": 2,
            "backoff_ms": 5000,
            "duration_ms": 920,
        },
    },
)
```

Chapter 5

Errors and Exceptions

Errors are valuable when they include context. A stack trace without IDs and fields is just a puzzle.

5.1 Log at the Boundary

Log failures where the system meets the outside world: request handlers, job runners, and queue consumers. That is where you can record the event name, the outcome, and the key fields in one place.

5.2 Avoid Duplicate Noise

If an error is logged with full context at the boundary, avoid logging the same error again at lower levels unless you add meaningfully different data.

5.3 Checklist: Error Logging

- Event name and outcome are present.
- A stable ID is included.
- The error type is captured in a field.
- Duration and attempt count are included when relevant.

5.4 Python Example

```
import logging

logger = logging.getLogger("importer")

try:
```

```
    raise ValueError("schema mismatch")
except Exception as exc:
    logger.error(
        "record.validate: outcome=failure",
        extra={
            "event_name": "record.validate",
            "level": "error",
            "outcome": "failure",
            "ids": {"batch_id": "batch_44"},
            "fields": {
                "error_type": type(exc).__name__,
                "duration_ms": 12,
                "rows_in": 1,
                "rows_out": 0,
            },
        },
    )
```

Chapter 6

Safety, Privacy, and Compliance

Logs are data. Treat them as production data with risk. The easiest security incident to avoid is the one you never log.

6.1 What Not to Log

Never log secrets or full tokens. Avoid personal data unless it is required and approved. Use opaque IDs instead of names or emails.

6.2 Redaction and Sampling

Redaction should happen before logging, not in the log backend. Sampling is useful for high-volume debug events, but it must keep the level and outcome accurate.

6.3 Checklist: Safe Logging

- Fields are reviewed for sensitive values.
- IDs are opaque and stable.
- Sampling does not drop error events.
- Retention matches legal requirements.

6.4 Table: Risk Examples

Risk	Safer alternative
Email address in fields	Use a user id.
Access token in message	Do not log; store in secret manager.
Full payload on error	Log a payload hash and key fields.

Chapter 7

AWS Lambda Correlation IDs

Serverless functions are short-lived and often noisy. You need correlation to connect events across services. This chapter focuses on AWS Lambda and how to keep IDs consistent.

7.1 Request ID vs Correlation ID

The Lambda `request_id` is unique per invocation. A `correlation_id` should link multiple services and retries. Use both. The request id tells you which execution produced the event. The correlation id tells you which larger operation it belongs to.

7.2 ContextVar and Filter

Use a context variable to hold the correlation id. A logging filter can inject it into every record.

```
import json
import logging
from contextvars import ContextVar

correlation_id = ContextVar("correlation_id", default="unknown")

class CorrelationIdFilter(logging.Filter):
    def filter(self, record: logging.LogRecord) -> bool:
        record.correlation_id = correlation_id.get()
        return True

logger = logging.getLogger("lambda")
logger.addFilter(CorrelationIdFilter())
```

7.3 Minimal JSON Formatter

A small formatter keeps logs structured without heavy dependencies.

```
class JsonFormatter(logging.Formatter):
    def format(self, record: logging.LogRecord) -> str:
        payload = {
            "event_name": getattr(record, "event_name", record.getMessage().split()[0]),
            "level": record.levelname.lower(),
            "outcome": getattr(record, "outcome", "unknown"),
            "ids": {
                "request_id": getattr(record, "request_id", "unknown"),
                "correlation_id": getattr(record, "correlation_id", "unknown"),
            },
            "fields": getattr(record, "fields", {}),
            "message": record.getMessage(),
        }
        return json.dumps(payload)
```

7.4 Putting It Together

```
import logging

def handler(event, context):
    correlation_id.set(event.get("correlation_id", "unknown"))
    logger.info(
        "job.start: outcome=success",
        extra={
            "event_name": "job.start",
            "outcome": "success",
            "request_id": context.aws_request_id,
            "fields": {"attempt": 1},
        },
    )
    return {"status": "ok"}
```

Chapter 8

Message Catalogue for Data Pipelines

Data pipelines need clear, repeatable messages. The catalogue below focuses on batch jobs, HTTP calls, queues, storage writes, and validation.

8.1 Batch Jobs

Event name	Fields
`batch.start`	`job_id`, `rows_in`, `attempt`
`batch.completed`	`rows_in`, `rows_out`, `duration_ms`
`batch.failed`	`error_type`, `rows_in`, `attempt`

8.2 HTTP Calls

Event name	Fields
`http.call`	`url_host`, `status_code`, `duration_ms`, `attempt`
`http.retry`	`backoff_ms`, `attempt`, `status_code`

8.3 Queue Processing (SQS-like)

Event name	Fields
`queue.receive`	`queue_name`, `message_id`, `attempt`
`queue.processed`	`message_id`, `duration_ms`, `rows_out`
`queue.failed`	`message_id`, `error_type`, `backoff_ms`

8.4 Storage Writes (DynamoDB or Iceberg)

Event name	Fields
`storage.write`	`table`, `rows_out`, `duration_ms`
`storage.commit`	`table`, `attempt`, `duration_ms`
`storage.failed`	`table`, `error_type`, `attempt`

8.5 Validation and Data Quality

Event name	Fields
`validation.start`	`rule_set`, `rows_in`
`validation.failed`	`rule`, `rows_out`, `outcome`
`validation.summary`	`rows_in`, `rows_out`, `duration_ms`

8.6 Checklist: Catalogue Use

- Event name is consistent across jobs.
- Fields match the table entries.
- IDs are present for traceability.
- Outcome matches the level.

Chapter 9

Testing and Review

Log messages are part of your operational API. Tests keep them stable. Review keeps them readable.

9.1 pytest caplog Basics

```
import logging

def test_log_message(caplog):
    logger = logging.getLogger("worker")
    with caplog.at_level(logging.INFO):
        logger.info(
            "queue.processed: outcome=success",
            extra={
                "event_name": "queue.processed",
                "outcome": "success",
                "ids": {"message_id": "m-9"},
                "fields": {"duration_ms": 12, "rows_out": 3},
            },
        )

    record = caplog.records[0]
    assert record.event_name == "queue.processed"
    assert record.outcome == "success"
```

9.2 Checking Required Fields

```
import logging

def test_required_fields(caplog):
    logger = logging.getLogger("pipeline")
    with caplog.at_level(logging.WARNING):
        logger.warning(
            "batch.load: outcome=retry",
            extra={
```

```
            "event_name": "batch.load",
            "outcome": "retry",
            "fields": {"attempt": 2, "backoff_ms": 1000},
            "ids": {"job_id": "job_22"},
        },
    )

record = caplog.records[0]
assert "job_id" in record.ids
assert record.fields["attempt"] == 2
```

9.3 Checklist: Review and Tests

- Tests assert event name and outcome.
- Required IDs are present.
- Fields include duration and attempt when relevant.
- Levels match the outcome.

Appendix A

Quick Reference

This appendix summarizes essential practices for fast recall.

A.1 Do

- Write messages that describe event name and outcome.
- Use consistent levels across services.
- Include correlation IDs with every request log.
- Keep field names stable and documented.

A.2 Avoid

- Logging secrets or personal data.
- Duplicating the same error at multiple layers.
- Vague messages like `"failed"` without fields.
- High-volume debug logs in production.

A.3 Quick Checklist

- Event name present.
- Outcome explicit.
- Level matches outcome.
- IDs and fields included.